DTI201 • บทเรียนแยกก่อน Lab MongoDB

จาก Container สู่ Docker และ Docker Compose

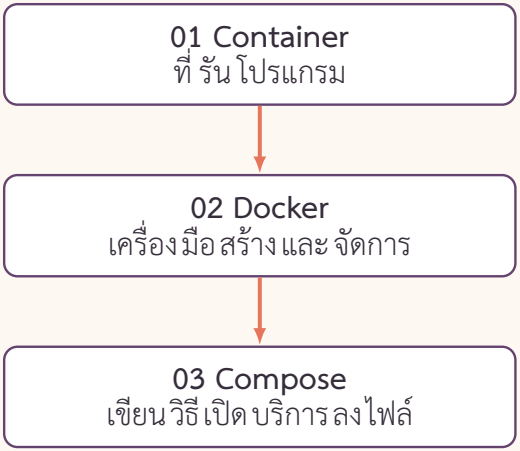
เข้าใจสิ่งที่กำลังรัน ก่อนพิมพ์คำสั่ง



บทเรียนและแล็บสำหรับผู้เริ่มต้น • Container → Docker → Compose

วันนี้จะเรียนตามเส้นทางใด?

1. **Container**: ทำไมเราต้องจัดเตรียมที่รันโปรแกรม และ container มีอะไรอยู่ข้างใน
2. **Docker**: image คืออะไร และใช้ Docker สร้าง เปิด ดู และหยุด container อย่างไร
3. **Docker Compose**: เขียนไฟล์บอกบริการ พอร์ต และที่เก็บข้อมูล แล้วใช้คำสั่งเปิดบริการ



ลองคิดหรือทดลอง: ข้อใดคือสิ่งที่กำลังรัน และข้อใดคือเครื่องมือที่ใช้จัดการ?

ช่วงที่ 1 / 3

Container คืออะไร?

เริ่มจากปัญหาของโปรแกรม ก่อนรู้จักคำสั่ง

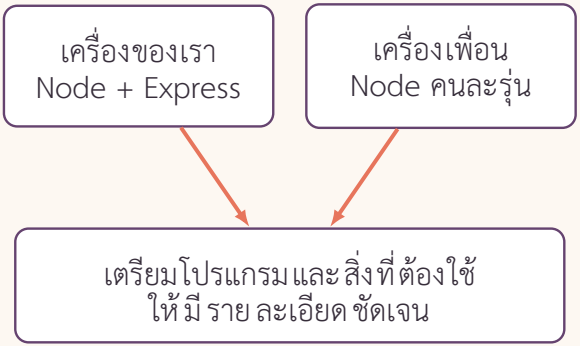
Container → Docker → Docker Compose

ทำไมโปรแกรมเดียวกันจึงเปิดไม่เหมือนกัน?

สมมติเราเขียนเว็บที่ต้องใช้ Node.js และแพ็กเกจ Express. เครื่องเพื่อนอาจมี Node คนละรุ่น หรือยังไม่มีแพ็กเกจที่โปรแกรมต้องใช้

เราจึงต้องบอกให้ชัดว่าโปรแกรมต้องใช้อะไร และเตรียมสภาพแวดล้อมให้รันได้

Container เป็นวิธีหนึ่งในการจัดโปรแกรมกับสิ่งที่ต้องใช้ไว้เป็นชุดสำหรับนำไปรัน



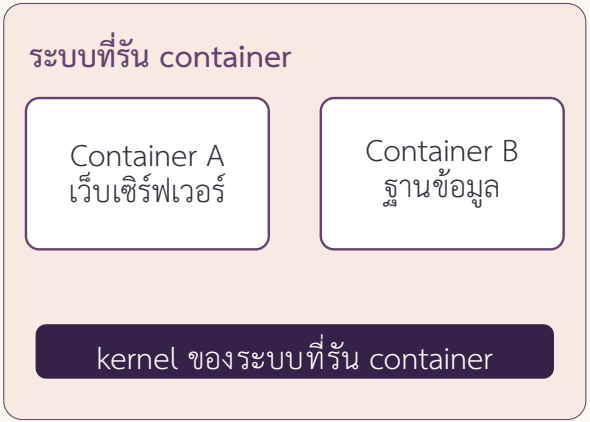
ลองคิดหรือทดลอง: ถ้าโปรแกรมเปิดไม่ได้บนเครื่องเพื่อน เราควรตรวจอะไรเป็นอย่างแรก?

Container คือโปรแกรมที่กำลังรันในพื้นที่แยก

Container คือกระบวนการที่กำลังรัน โดยมีพื้นที่มองเห็นไฟล์ เครือข่าย และทรัพยากรที่แยกจากกระบวนการอื่นตามการตั้งค่า

ใน container อาจมีเว็บเซิร์ฟเวอร์หนึ่งตัว. บนเครื่องเดียวกันเราสร้างอีก container สำหรับฐานข้อมูลได้

Container ใช้ kernel ของระบบที่รัน container; มันไม่ใช่เครื่องคอมพิวเตอร์เต็มเครื่องแยกอีกเครื่อง



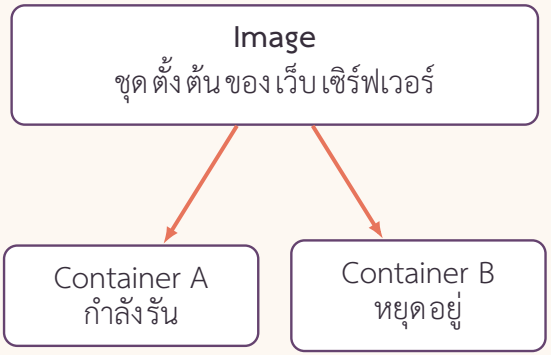
ลองคิดหรือทดลอง: ถ้าเว็บหยุดทำงาน ฐานข้อมูลที่อยู่ในอีก container จำเป็นต้องหยุดด้วยหรือไม่?

Image กับ container ต่างกันอย่างไร?

Image คือชุดไฟล์และข้อมูลตั้งต้นที่ใช้สร้าง container เช่น image ของเว็บเซิร์ฟเวอร์

Container คือสิ่งที่สร้างจาก image และอาจกำลังรันหรือหยุดอยู่ เราสร้างหลาย container จาก image เดียวกันได้

นึกถึง image เป็นแบบสำหรับสร้าง และ container เป็นแต่ละชุดที่สร้างขึ้นจริง



ลองคิดหรือทดลอง: ลบ container A แล้ว image ที่ใช้สร้างยังอยู่หรือไม่? จะตรวจอย่างไรในช่วงถัดไป?

ช่วงที่ 2 / 3

Docker ช่วยจัดการ Container

รู้จักเครื่องมือก่อนใช้คำสั่งแรก

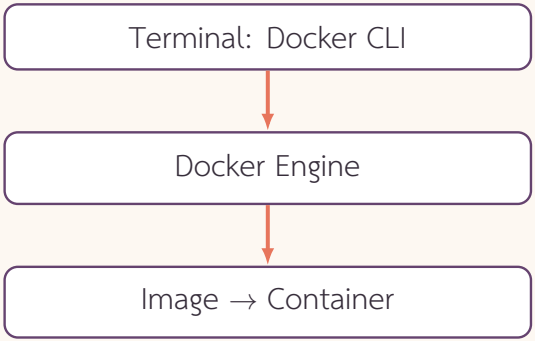
Container → Docker → Docker Compose

Docker คืออะไร?

Docker เป็นชุดเครื่องมือสำหรับสร้าง แจกจ่าย และรัน container

ในเครื่องของเรา **Docker CLI** รับคำสั่งที่พิมพ์ใน terminal แล้วส่งไปยัง **Docker Engine** ซึ่งจัดการ image และ container

Image อาจมาจาก registry ซึ่งเป็นแหล่งเก็บ image เช่น Docker Hub. ก่อนทำแล็บให้เปิด Docker Engine และตรวจว่าใช้งานได้



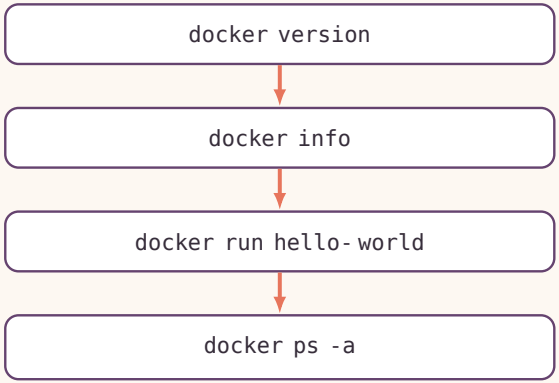
ลองคิดหรือทดลอง: ถ้า CLI ตอบว่าเชื่อม Docker Engine ไม่ได้ ควรตรวจอะไร?

ตรวจเครื่อง แล้วรัน container แรก

`docker version` แสดงรุ่นของ client และ server. `docker info` ช่วยตรวจว่า CLI ติดต่อ Engine ได้

จากนั้นรัน `docker run hello-world`. Docker หา image ในเครื่อง; ถ้ายังไม่มีจะพยายามดาวน์โหลด แล้วสร้าง container เพื่อแสดงข้อความ

Container ตัวนี้ทำงานสั้น ๆ แล้วหยุด. ใช้ `docker ps -a` ดูรายการที่หยุดแล้วด้วย



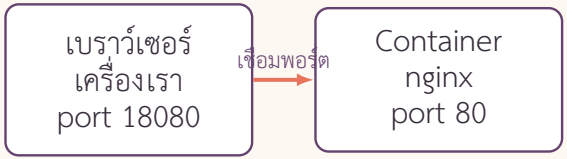
ลองคิดหรือทดลอง: ทำไม `docker ps` อาจไม่แสดง hello-world ที่เพิ่งรันเสร็จ?

เปิดเว็บใน container แล้วเข้าจากเบราว์เซอร์

Image `nginx:alpine` มีเว็บเซิร์ฟเวอร์. เราสร้าง container ชื่อ `dti201-web` ให้รันเบื้องหลังด้วย `-d`

`-p 127.0.0.1:18080:80` เชื่อม port 18080 บนเครื่องเราไปยัง port 80 ภายใน container

เปิด `http://localhost:18080` หรือใช้ curl แล้วดูผล. ที่อยู่ 127.0.0.1 จำกัดการเข้าถึงจากเครื่องตนเอง




```
-p 127.0.0.1:18080:80
```

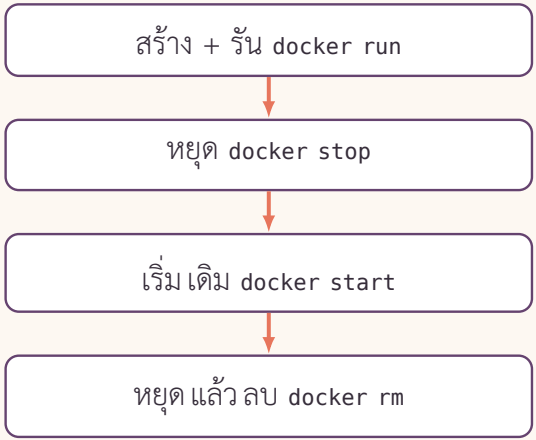
ลองคิดหรือทดลอง: เลข 18080 กับ 80 อยู่คนละที่อย่างไร?

ดูสถานะและหยุดเฉพาะ container ของเรา

`docker ps` แสดง container ที่กำลังรัน. `docker logs dti201-web` ดูข้อความจากเว็บเซิร์ฟเวอร์

`docker stop dti201-web` หยุด container; `docker start dti201-web` เปิด container เดิมอีกครั้ง

`docker rm dti201-web` ลบ container ที่หยุดแล้ว. Image ยังอยู่ และสามารถใช้สร้าง container ใหม่ได้



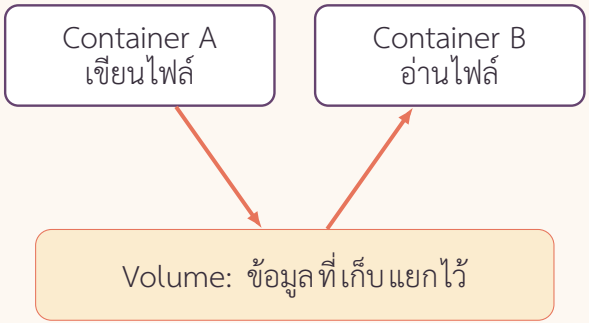
ลองคิดหรือทดลอง: หลังหยุด container หน้าเว็บ port 18080 ยังตอบหรือไม่? ลองแล้วจดผล

ถ้าลบ container ข้อมูลอยู่ที่ไหน?

ไฟล์ที่เขียนไว้ในชั้นเขียนของ container อาจหายเมื่อ container ถูกลบ

Volume เป็นพื้นที่เก็บข้อมูลที่ Docker จัดการแยกจากวงจรชีวิตของ container. เราเชื่อม volume เข้ากับตำแหน่งภายใน container ได้

แล็บจะเขียนข้อความหนึ่งบรรทัดใน volume แล้วเปิด container ใหม่มาอ่าน เพื่อดูว่าข้อมูลยังอยู่



ลองคิดหรือทดลอง: ถ้าหยุดและลบ container A แล้ว container B ยังอ่านไฟล์ใน volume เดิมได้หรือไม่?

ช่วงที่ 3 / 3

Docker Compose คืออะไร?

จากคำสั่งยาว ๆ สู่ไฟล์ที่อธิบายบริการ

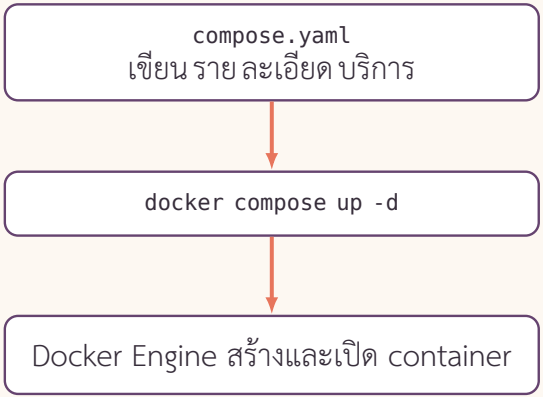
Container → Docker → Docker Compose

ทำไมต้องมี Compose?

เมื่อแอปมีหลายส่วน เช่นเว็บและฐานข้อมูล เราต้องจำ image, port และ volume ของแต่ละส่วน

Docker Compose ให้เราเขียนรายละเอียดบริการไว้ใน `compose.yaml`. จากนั้น `docker compose up -d` ใช้ไฟล์นี้สร้างและเริ่มบริการ

เริ่มจากไฟล์ที่มีบริการเดียวก่อน แล้วอ่านไฟล์ MongoDB ของ Lab 10



ลองคิดหรือทดลอง: Compose อ่านรายละเอียดจากไฟล์ใด และ Docker Engine ทำอะไรต่อ?

อ่านไฟล์ Compose ทีละบรรทัด

`services` รวมบริการที่ต้องการเปิด; `web` เป็นชื่อบริการที่เราตั้ง

`image` ระบุชุดตั้งต้น; `ports` ระบุ port เครื่องเราและ port ใน container

ในแล็บเราจะสร้างไฟล์นี้เอง แล้วเปิด `localhost:18081` เพื่อดูผล

```
services:
  web:
    image: nginx:alpine
    ports:
      - "127.0.0.1:18081:80"
```

18081 บนเครื่อง → 80 ใน container

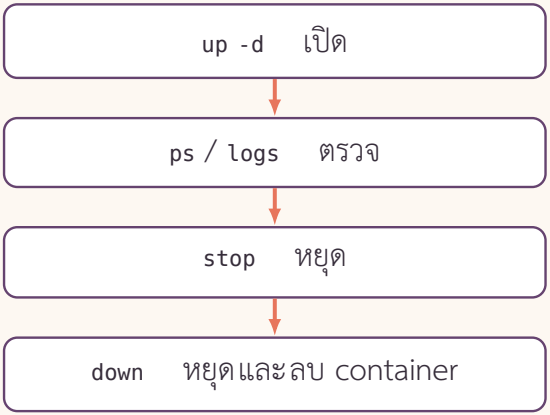
ลองคิดหรือทดลอง: ถ้าเข้า port 18081 ไม่ได้ จะดูสถานะบริการด้วยคำสั่งใด?

คำสั่ง Compose ที่ใช้ในคาบ

`docker compose up -d` สร้างและเปิดบริการตามไฟล์

`docker compose ps` ดูสถานะ และ `docker compose logs` ดูข้อความจากบริการ

`docker compose stop` หยุด container เดิม; `docker compose down` หยุดและลบ container กับ network ที่ Compose สร้าง โดย named volume ยังอยู่ตามค่าเริ่มต้น



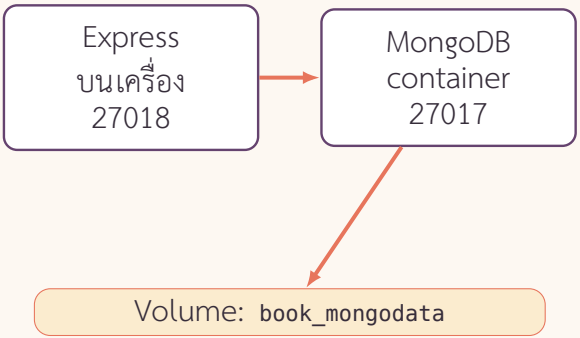
ลองคิดหรือทดลอง: หยุด container กับลบ container ต่างกันอย่างไร?

เชื่อมกลับไป Lab 10: MongoDB

ในไฟล์ Compose ของ Lab 10 บริการชื่อ `db` ใช้ image `mongo:7.0`

`127.0.0.1:27018:27017` ให้ Express บนเครื่องติดต่อ MongoDB ผ่าน port 27018; MongoDB ฟัง port 27017 ภายใน container

`book_mongodata:/data/db` เชื่อม named volume กับที่ MongoDB เก็บไฟล์. การเปิด container ไม่ได้แทนขั้นตอนเขียน Express API



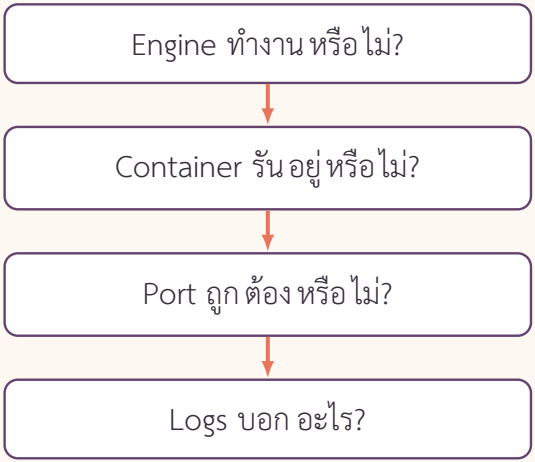
ลองคิดหรือทดลอง: port ใดอยู่บนเครื่องเรา และ volume เก็บข้อมูลส่วนใด?

เมื่อเจอปัญหา ให้ตรวจทีละจุด

ถ้า `docker info` ต่อไม่ได้ ให้ตรวจว่า Engine เปิดอยู่

ถ้า port ถูกใช้อยู่ ให้ดูข้อความผิดพลาดแล้วเลือก port ฝั่งเครื่องตัวอื่น; ถ้าดาวน์โหลด image ไม่ได้ ให้ตรวจอินเทอร์เน็ตหรือขอความช่วยเหลือ

ถ้า container หยุดทันที ให้ดู `docker ps -a` และ `docker logs`. อย่าแก้ปัญหาด้วยการลบข้อมูลทั้งหมด



ลองคิดหรือทดลอง: ถ้า `docker ps` ไม่เห็นเว็บ จะใช้คำสั่งใดดู container ที่หยุดไปแล้ว?

สิ่งที่ต้องแสดงหลังจบแล็บ

1. อธิบาย image และ container โดยใช้ตัวอย่าง nginx ของตน
2. เปิดหน้าเว็บจาก port 18080 แล้วหยุดและเปิด container เดิม
3. เขียนและอ่านข้อความจาก volume ผ่าน container คนละตัว
4. สร้าง `compose.yaml`, เปิดบริการที่ port 18081 แล้วอธิบายไฟล์ MongoDB ของ Lab 10

Container: อะไรเป็นตัวรัน?

Docker: คำสั่งใดสร้างและหยุด?

Compose: ไฟล์ใดบอกบริการ?

ลองคิดหรือทดลอง: ก่อนเริ่ม Lab 10 ให้วาดเส้นทาง Express → port → MongoDB → volume

อ่านเพิ่มเติม

Docker Docs: What is a container?

Docker Docs: Run your first container

Docker Docs: Docker Compose Quickstart

Docker Docs: docker compose down และ volumes

ทำตามใบงาน Docker แยก แล้วจึงกลับไป Lab 10 MongoDB